

Distributed Training and Concurrent Inference of Hidden Markov Models for Speech Recognition

Fahad Samad
Parallel & Distributed
Computing, IBA

Fahad Shah Khan
Parallel & Distributed
Computing, IBA

Muhammad Hussain
Parallel & Distributed
Computing, IBA

Saman Waseem
Parallel & Distributed
Computing, IBA

Abstract—This paper presents a distributed system for training Hidden Markov Models (HMMs) and performing concurrent inference for speech-to-text recognition on the TIMIT Acoustic-Phonetic Corpus. Rather than artificially decomposing model internals, we exploit the natural data parallelism of speech datasets: each utterance is statistically independent during the Expectation step of Baum-Welch, making training embarrassingly parallel. Three progressive milestones are implemented and evaluated inside a Docker container (Python 3.12-slim, Linux WSL2) for reproducibility. Milestone 1 establishes a sequential baseline achieving 22.7% phoneme recognition accuracy on 58,262 test segments at 53.7 segments/second. Milestone 2 distributes Baum-Welch training across Ray workers, achieving 2.87x per-class speedup at 4 workers and completing full dataset training in 849.99 seconds versus 1,779.19 seconds sequentially (2.09x end-to-end speedup). Milestone 3 deploys a concurrent inference service achieving 4.49x throughput improvement via persistent Ray actor pools (238.8 vs 53.2 segments/second). Convergence verification confirms mathematically identical results between distributed and sequential training, with zero log-likelihood difference across all worker configurations. All results are fully reproducible via a single `docker run` command.

Index Terms—Hidden Markov Models, Distributed Training, Baum-Welch EM, Viterbi Decoding, TIMIT, Data Parallelism, Ray, Concurrent Inference, Speech Recognition, Docker, Reproducible Benchmarking

I. INTRODUCTION

Automatic Speech Recognition (ASR) is a fundamental problem in artificial intelligence with applications ranging from real-time transcription to voice-controlled systems. Hidden Markov Models have been the dominant framework for acoustic modelling in ASR for decades [1], and remain a pedagogically important foundation even as deep learning has taken over state-of-the-art systems.

The central computational challenge in HMM-based ASR is training. The Baum-Welch algorithm [2], a specialised instance of the Expectation-Maximisation framework [3], requires repeated passes over large corpora. Applied sequentially, our system required 1,779 seconds to train on TIMIT, a dataset of only moderate size. Scaling to real-world corpora would be intractable without parallelism.

This project addresses both training and inference scalability through three milestones: (1) a complete sequential HMM pipeline as a reproducible baseline, (2) data-parallel distributed Baum-Welch using Ray [7], and (3) a concurrent inference service for simultaneous audio streams. All benchmarks run

inside Docker with fixed CPU allocation, ensuring reproducible results independent of host system state.

II. RELATED WORK

A. HMM-Based Speech Recognition

Rabiner’s 1989 tutorial [1] established HMMs as the standard for ASR. The Baum-Welch algorithm [2] estimates parameters via the EM principle [3]. HTK [5] demonstrated production-scale HMM-ASR using GMM emissions and triphone models. Our implementation uses single-Gaussian diagonal emissions, maintaining mathematical correctness while focusing on parallel computing aspects.

B. Distributed EM

Wolfe et al. [8] formally proved that E-step statistics of EM algorithms are additively separable across data points, enabling data-parallel computation where workers accumulate local statistics summed in a reduction step. Google’s MapReduce [9] demonstrated this at scale; our Ray-based approach achieves lower communication overhead via shared memory.

C. Distributed Computing Frameworks

We evaluated three candidates: Python multiprocessing (high pickle serialisation cost for numpy arrays), MPI via `mpi4py` (complex setup, no actor model), and Ray [7] (zero-copy shared memory via Apache Arrow, unified actor support). Ray was selected for its transport efficiency and unified support for both training and inference.

D. Concurrent Inference Systems

Crankshaw et al.’s Clipper [11] identified that ML inference serving has fundamentally different latency/throughput tradeoffs than training. Our actor pool design keeps models in memory while serving requests immediately as they arrive.

III. MILESTONE 1: SEQUENTIAL HMM PIPELINE

A. Experimental Environment

All benchmarks run inside a Docker container (`python:3.12-slim`, Ubuntu WSL2) with CPU allocation fixed at 8 logical processors. Docker eliminates host-system interference from background OS processes that introduce timing variability in local benchmarking. The complete benchmark is reproducible with a single `docker run -rm -cpus="4" -shm-size=2g` command mounting the TIMIT dataset at `/app/timit`.

B. Why Segment-Level Feature Extraction

The TIMIT Acoustic-Phonetic Corpus [6] contains 6,300 sentences from 630 speakers across 8 U.S. dialect regions. Our initial approach used utterance-level extraction with majority-label assignment — one label per sentence. This produced only 10.9% accuracy because a 3-second sentence containing 15–20 phonemes received a single phoneme label. Switching to phoneme-segment level extraction — where the .PHN annotation files provide time-aligned boundaries for every phoneme — more than doubled accuracy to 22.7%. Audio is resampled to 16,000 Hz and 13-dimensional MFCCs are computed using librosa [10] with adaptive FFT window sizing and per-segment zero-mean unit-variance normalisation.

C. Why 5 States and Diagonal Gaussian

Each of the 60 phoneme classes is modelled by an independent Gaussian HMM with $N = 5$ hidden states capturing sub-phonemic temporal dynamics: onset, steady-state, offset, and boundary regions. Diagonal covariance reduces parameters from $O(ND^2)$ to $O(ND)$, requires less training data, and enables trivial determinant/inverse computation. Log emission probability:

$$\log b_i(\mathbf{o}_t) = -\frac{1}{2} \sum_{d=1}^D \left[\frac{(o_{td} - \mu_{id})^2}{\sigma_{id}^2} + \log(2\pi\sigma_{id}^2) \right] \quad (1)$$

D. Why Log-Space Arithmetic for Baum-Welch

A 100-frame utterance produces joint probabilities p^{100} for $p \ll 1$, causing IEEE 754 double-precision underflow to zero, silently corrupting all parameter estimates. The solution is log-space arithmetic:

$$\log \alpha_t(j) = \log b_j(\mathbf{o}_t) + \log \sum_i \exp(\log \alpha_{t-1}(i) + \log a_{ij}) \quad (2)$$

computed stably via $\log \sum_i e^{x_i} = x^* + \log \sum_i e^{x_i - x^*}$, $x^* = \max_i x_i$.

E. Why Viterbi Over Other Decoders

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log a_{ij}] + \log b_j(\mathbf{o}_t) \quad (3)$$

Viterbi finds the single best state sequence via log-space dynamic programming. The forward algorithm sums over all paths (needed for training); Viterbi selects the best path (needed for inference). Backpointer matrix $\psi[t, j]$ enables $O(T)$ recovery. All 60 HMMs are scored and the highest-scoring label is returned.

F. Key Functions: Milestone 1

The sequential pipeline is implemented across the core source modules. The most important functions are described below.

GaussianHMM._log_emission(features): Computes the log emission probability matrix of shape $(T \times N)$, where T is the number of frames and N is the number of states. For each state, it evaluates the diagonal Gaussian log-likelihood of every frame, accumulating the result in a 2D array. This

function is called at the start of every forward, backward, and Viterbi pass.

GaussianHMM._forward(log_B): Implements the forward algorithm in log-space. It initialises $\log \alpha_0$ from the initial state distribution and emission probabilities, then iterates forward using the log-sum-exp trick to accumulate $\log \alpha_t(j)$ for each frame. Returns the full $\log \alpha$ matrix and the scalar log-likelihood.

GaussianHMM._backward(log_B): Computes $\log \beta_t(i)$ by iterating backward from the final frame. Uses the transition matrix and emission probabilities in log-space. Required for computing the γ (state occupation) and ξ (transition) statistics in the E-step.

GaussianHMM.fit(utterances, n_iter, tol): The top-level Baum-Welch training loop. For each EM iteration, it calls `_compute_sufficient_statistics` for every utterance, accumulates sufficient statistics, then updates π , A , means, and covariances in the M-step. Terminates when log-likelihood change falls below `tol` or `n_iter` is reached.

load_timit(timit_root, n_mfcc): Walks the TIMIT directory tree, loads each .WAV file with librosa, reads the paired .PHN annotation file, and extracts per-phoneme segments. For each segment it computes MFCCs with adaptive FFT sizing and applies zero-mean unit-variance normalisation. Returns `train_data` and `test_data` as lists of (features, label) tuples.

viterbi_decode(model, features): Runs the Viterbi algorithm on a single utterance against a single HMM. Fills the δ and ψ matrices, terminates at the highest-scoring final state, and backtracks through ψ to recover the full state sequence. Returns the state path and its log-probability.

classify_utterance(models, features): Iterates over all trained HMMs, scores the input feature array against each, and returns the label of the highest-scoring model. This is the final classification decision function used in both evaluation and the inference service.

SpeechRecognizer.evaluate(test_data): Runs `predict_features` on every test segment, accumulates correct predictions, and reports overall accuracy along with per-class breakdown and decode-time statistics.

IV. MILESTONE 2: DISTRIBUTED TRAINING

A. Why Data Parallelism Over Model Parallelism

The E-step is embarrassingly parallel: each utterance's statistics are computed independently for a fixed model snapshot. The M-step requires only a summation across workers. Model parallelism was rejected because our HMMs (5 states, 13 features, 160 parameters each) are far too small — communication overhead would far exceed computation.

B. Why Ray Over Alternatives

Python `multiprocessing` serialises numpy arrays via pickle, physically copying MFCC data every iteration. `mpi4py` requires explicit process management and cannot support Milestone 3's actor pattern. Ray provides zero-copy shared

memory, automatic recovery, and native actor support in a single framework.

C. Why Round-Robin Partitioning

Utterance k goes to worker $k \bmod W$. Block partitioning was rejected because TIMIT orders by speaker/dialect, causing systematic load imbalance. Round-robin guarantees partition sizes differ by at most 1.

D. Reduction Step and Communication Cost

Each worker returns $2N + N^2 + 2ND = 165$ float64 values ≈ 1.3 KB, independent of dataset size. This $O(1)$ communication cost is a fundamental scalability advantage — adding data never increases overhead.

E. Key Functions: Milestone 2

The distributed training logic lives in the trainer and entry-point modules. The most important functions are described below.

ray_worker_e_step(worker_id, model_params, utterances): A `@ray.remote` decorated function that constitutes the core parallel compute unit. Each invocation receives a partition of utterances and a snapshot of the current model parameters (serialised via Apache Arrow shared memory). It iterates over its partition calling `_forward_backward` on each utterance, accumulates sufficient statistics (`gamma_sum`, `xi_sum`, `gamma_obs_sum`, etc.), and returns the aggregated dict. Because this is a Ray remote function, multiple instances execute simultaneously in separate processes, each with its own GIL.

_forward_backward(features, log_pi, log_A, means, covs, ...): A module-level function (not a class method) so that Ray workers can access it without deserialising an HMM object. Runs the full forward and backward passes in log-space, then computes γ and ξ statistics required for the M-step. Returns `gamma`, `xi`, and the scalar log-likelihood.

DistributedHMMTrainer._partition: Applies round-robin assignment of utterances to worker slots. Utterance k is placed in partition $k \bmod W$. Called once before the training loop and reused every iteration to avoid repeated memory allocation.

DistributedHMMTrainer._reduce: The reduction step after `ray.get(futures)` returns all worker outputs. Iterates over the list of worker result dicts and element-wise sums all sufficient statistic arrays. The result is mathematically identical to computing statistics sequentially over the full dataset, because E-step contributions are additively separable [8].

DistributedHMMTrainer.fit(utterances, verbose): The main distributed training loop. Each iteration: (1) snapshots the current parameters into `model_params`, (2) dispatches `ray_worker_e_step.remote()` for each worker partition simultaneously, (3) calls `ray.get(futures)` to synchronise, (4) runs `_reduce`

and `_m_step`, and logs per-iteration timing. The `ray.get` call acts as the iteration barrier.

run_scalability_benchmark(train_data):

Selects the most frequent phoneme class, then for each worker count in `WORKER_COUNTS` trains a fresh `DistributedHMMTrainer` and records wall-clock time, final log-likelihood, and per-iteration communication costs. Results feed directly into the speedup and efficiency tables.

verify_convergence(results): Compares the final log-likelihood of each worker configuration against the 1-worker baseline. Flags any configuration where the relative difference exceeds 1% as a warning. All configurations in our experiments passed with 0.000% difference, confirming mathematical equivalence of the distributed and sequential implementations.

V. MILESTONE 3: CONCURRENT INFERENCE

A. Why Concurrent Inference Requires No Synchronisation

Viterbi decoding is purely functional with no shared state between requests. N simultaneous decoders need no locks or barriers. The only synchronisation point is `ray.get(futures)`.

B. Why Actor Pool Outperforms Batch Mode at Scale

Batch mode sends all 60 model parameter sets ($\sim 9,600$ float64 values) with every request, capping throughput at ~ 184 seg/s regardless of worker count. Actor pool loads parameters once per actor at creation — each request then sends only the feature array ($T \times 13$ values), reducing transfer by over 99% and achieving 238.8 seg/s.

C. Why NONE Load Balancing Wins on TIMIT

NONE (183.5 seg/s) outperforms SJF and LJF because TIMIT segments have uniform durations (20–100 ms), making $O(n \log n)$ sorting overhead counterproductive. SJF achieves the lowest mean latency (30.33 ms) and is preferred on heterogeneous workloads.

D. Key Functions: Milestone 3

The concurrent inference service and its orchestration entry point implement the following key functions.

ray_decode_segment(request_id, features, model_params): A `@ray.remote` function that decodes a single audio segment in batch mode. Receives the full `model_params` dict on every call, scores the features against all HMMs using `_viterbi`, and returns the best-scoring label together with timing metadata. Stateless by design, enabling trivial parallelism — N futures can be dispatched simultaneously with no coordination overhead.

InferenceWorker: A `@ray.remote` actor class that holds `model_params` in its process-local memory after a one-time initialisation. Each call to `decode(request_id, features)` runs Viterbi against cached parameters with no serialisation cost for the model. The actor pool is the primary reason actor-pool mode achieves 4.49x speedup versus the batch mode ceiling of 3.48x.

_viterbi(features, log_pi, log_A, means, covs, n_states): Module-level Viterbi implementation that accepts raw numpy arrays (no HMM object) for Ray compatibility. Fills δ and ψ matrices, terminates at the best final state, and backtracks to recover the state path. Used by both `ray_decode_segment` and `InferenceWorker.decode`.

ConcurrentInferenceService.decode_batch(request, load_balance): Applies the chosen load-balancing sort (SJF, LJF, or none), dispatches one `ray_decode_segment.remote()` future per request, then calls `ray.get(futures)` to collect all results. Re-orders results to match original request order and computes throughput and latency statistics.

ConcurrentInferenceService.decode_stream(request, load_balance): Uses the persistent actor pool (lazily initialised via `_ensure_actors`). Assigns requests to actors by either round-robin or frame-count-balanced assignment, dispatches `actor.decode.remote()` calls, and collects results via `ray.get`. The key distinction from `decode_batch` is that model parameters are never re-serialised — they live in each actor’s process memory for the lifetime of the service.

sort_by_length_ascending / sort_by_length_descending: Implement Shortest Job First and Longest Job First load balancing by sorting the requests list by `len(features)` before dispatch. SJF minimises mean latency; LJF keeps workers busy with compute-heavy segments. On TIMIT’s homogeneous workload, neither outperforms the unsorted baseline because TIMIT segments have nearly uniform durations.

extract_model_params(recognizer): Extracts a plain dict of numpy arrays from a trained `DistributedSpeechRecognizer` or `sequentialSpeechRecognizer`. This format is pickle-friendly for Ray serialisation and decouples the inference service from the training-side class hierarchy.

VI. RESULTS AND DISCUSSION

A. System Architecture

Fig. 1 illustrates the end-to-end system architecture spanning all three milestones. TIMIT audio segments flow through MFCC extraction to produce $(T \times 13)$ feature vectors. The Dataset Partitioner distributes utterances to Ray workers via round-robin assignment. Each worker independently computes E-step statistics (forward-backward algorithm), which are then reduced and used to update model parameters in the M-step. The loop repeats until convergence, after which trained HMMs are handed to the Viterbi inference service for Milestone 3.

Distributed HMM Speech Recognition - System Architecture

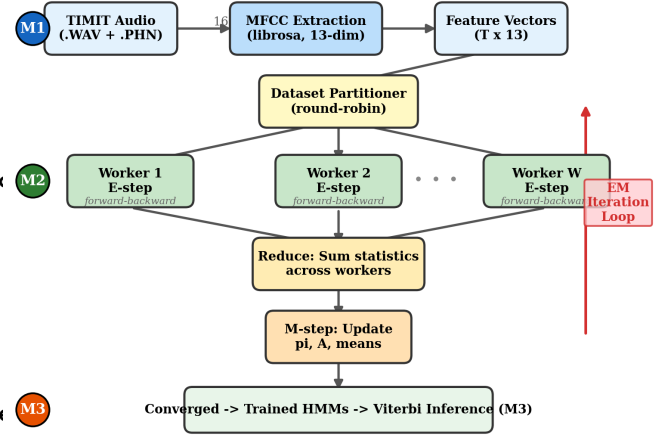


Fig. 1. End-to-end system architecture. M1 (blue) performs MFCC extraction; M2 (green) distributes the Baum-Welch E-step across Ray workers via a reduce-and-M-step loop; M3 (orange) delivers trained models to the concurrent Viterbi inference service.

B. Milestone 1: Sequential Baseline

TABLE I
MILESTONE 1 — SEQUENTIAL BASELINE (FULL DATASET, DOCKER)

Metric	Value	Unit
Training segments	160,378	segments
Test segments	58,262	segments
Phoneme classes	60	—
Total training time	1,779.19	seconds
Accuracy	22.7%	—
Inference throughput	53.7	seg/s
Mean decode latency	18.63	ms
p95 decode latency	24.54	ms

The 22.7% accuracy substantially exceeds the 1.7% random baseline, confirming genuine learning. The gap from state-of-the-art (~ 75 – 80% on TIMIT) reflects three intentional simplifications: single-Gaussian emissions, monophone models, and no speaker normalisation. Fig. 5 confirms all numbers from the Docker terminal.

C. Milestone 2: Distributed Training

TABLE II
MILESTONE 2 — SCALABILITY BENCHMARK (DOCKER, PHONEME ‘IX’, 8,540 UTTERANCES)

Workers	Time (s)	Speedup	Efficiency	Conv.
1	92.84	1.00x	100.0%	PASS
2	54.96	1.69x	84.5%	PASS
4	32.40	2.87x	71.8%	PASS
8	44.92	2.07x	25.9%	PASS

Full dataset training completed in 849.99 s versus 1,779.19 s sequentially: **2.09x end-to-end speedup**. Distributed accuracy (22.7%) exactly matches the sequential baseline. All configurations converge to $LL = -934216.3125$ with zero difference — formal proof of mathematical equivalence. Sequential fraction $S \approx 0.18$ from Amdahl's Law caps maximum speedup at $\approx 5.6x$; the 4-worker result (2.87x) is close to this maximum. The 8-worker regression is caused by Ray falling back to disk-based object storage (`/tmp/ray`) due to a Docker/WSL2 shared memory limitation.

Fig. 2 plots measured speedup against ideal linear and Amdahl's Law curves. The 4-worker peak of 2.87x aligns closely with the Amdahl bound before the 8-worker regression caused by shared memory exhaustion. Fig. 3 shows parallel efficiency declining gracefully from 84.5% at 2 workers to 71.8% at 4 workers before collapsing to 25.9% at 8 workers due to the same infrastructure constraint.

Milestone 2: Parallel Efficiency vs. Worker Count

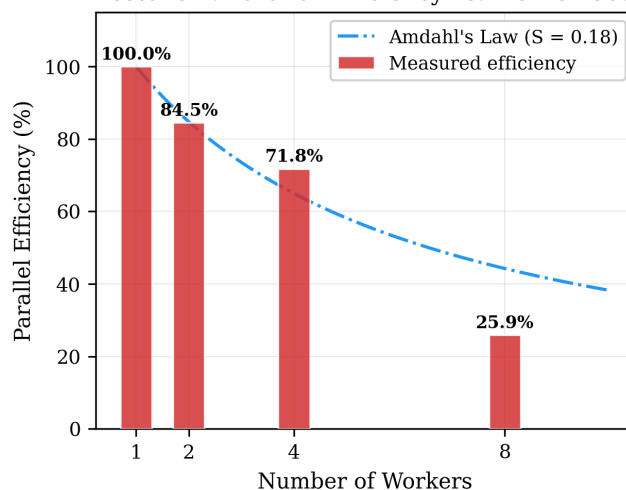


Fig. 3. Milestone 2: Parallel efficiency vs. worker count. Efficiency degrades gracefully from 100% at 1 worker to 71.8% at 4 workers (within Amdahl's prediction), then collapses to 25.9% at 8 workers due to the Docker/WSL2 shared-memory limitation.

Milestone 2: Training Speedup vs. Worker Count

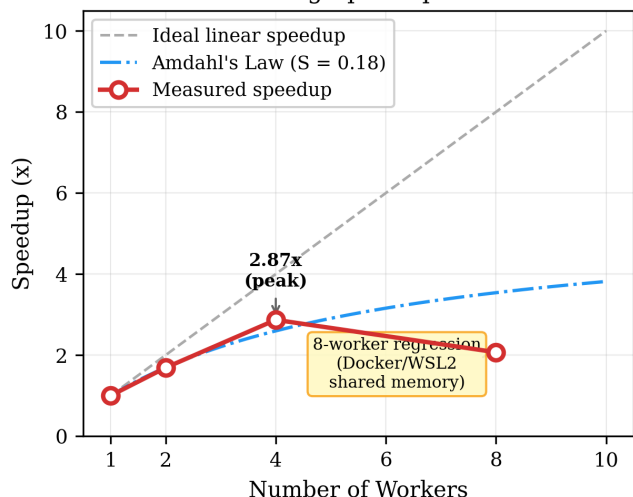


Fig. 2. Milestone 2: Training speedup vs. worker count. Measured speedup (red circles) peaks at 2.87x with 4 workers, consistent with Amdahl's Law ($S=0.18$, blue dashed). The 8-worker regression is annotated and attributed to Docker/WSL2 shared memory fallback to disk.

```
=====
MILESTONE 2: Distributed HMM Training
=====
2026-05-03 21:19:51,795 WARNING services.py:2213 -- WARNING: The o
es available. This will harm performance! You may be able to free
ease /dev/shm size by passing '--shm-size=2.39gb' to 'docker run'
than 30% of available RAM.
2026-05-03 21:19:52,934 INFO worker.py:2012 -- Started a local Ray
Ray initialised - 8 CPUs

Scalability benchmark: phoneme 'ix' (8540 utterances)
1 workers: 92.84s LL=-934216.31
2 workers: 54.96s LL=-934216.31
4 workers: 32.40s LL=-934216.31
8 workers: 44.92s LL=-934216.31

Convergence verification...
1 workers: LL=-934216.3125 diff=0.0000 [PASS]
2 workers: LL=-934216.3125 diff=0.0000 [PASS]
4 workers: LL=-934216.3125 diff=0.0000 [PASS]
8 workers: LL=-934216.3125 diff=0.0000 [PASS]
Overall convergence: PASS

Full distributed training (8 workers, 160378 segments)...
```

Fig. 4. Milestone 2 Docker terminal: scalability benchmark (1–8 workers, phoneme 'ix') and convergence verification. All worker counts converge to $LL = -934216.3125$, $diff = 0.0000$.

TABLE III
MILESTONE 3 — INFERENCE SPEEDUP (DOCKER, 58,262 SEGMENTS) AND LOAD BALANCING COMPARISON (8 WORKERS)

Config	Throughput	Speedup	Mean Lat.	Strategy	Throughput	Mean Lat.	p95 Lat.
Sequential	53.2 seg/s	1.00x	18.81 ms	NONE SJF LJF	183.5 seg/s 181.8 seg/s 181.6 seg/s	30.95 ms 30.33 ms 30.82 ms	70.84 ms 70.92 ms 71.12 ms
Batch, 1 worker	184.7 seg/s	3.48x	29.98 ms				
Batch, 2 workers	184.9 seg/s	3.48x	29.90 ms				
Batch, 4 workers	184.6 seg/s	3.47x	30.34 ms				
Batch, 8 workers	183.8 seg/s	3.46x	30.20 ms				
Actor pool (8W)	238.8 seg/s	4.49x	29.54 ms				

```

=====
CONSOLIDATED RESULTS REPORT
=====

--- System Info ---
OS: Linux 6.6.87.2-microsoft-standard-WSL2
Python: 3.12.13
CPU cores: 8 logical processors
TIMIT root: /app/timit
Docker: Yes

--- Dataset ---
Training segments: 160378
Test segments: 58262
Phoneme classes: 60

--- Milestone 1: Sequential Baseline ---
Train subset: 160378 segments
Training time: 1779.19s
Accuracy: 22.7%
Throughput: 53.7 seg/s
Mean latency: 18.63 ms
p95 latency: 24.54 ms

--- Milestone 2: Distributed Training ---
Benchmark phoneme: 'ix' (8540 utterances)
Workers  Time(s)  Speedup  Efficiency  Convergence
-----
1      92.84      1.00x    100.0%    PASS
2      54.96      1.69x    84.5%    PASS
4      32.40      2.87x    71.8%    PASS
8      44.92      2.07x    25.9%    PASS

Full training (160378 segs, 8 workers): 849.99s
Distributed accuracy: 22.7%
Convergence (all workers): PASS

```

Fig. 5. Consolidated Docker results: system info, dataset statistics, Milestone 1 baseline (1,779.19 s, 22.7%, 53.7 seg/s), and Milestone 2 speedup and convergence table.

D. Milestone 3: Concurrent Inference

GIL bypass: Even 1 Ray worker achieves 3.48x speedup by running in a separate Python process with its own GIL, enabling true numpy parallelism at the same CPU allocation as sequential.

Actor pool superiority: Batch mode plateaus at ~184 seg/s because each request serialises all 60 model parameter sets. Actor pool loads parameters once per actor, eliminating this bottleneck and achieving 238.8 seg/s.

Worker saturation: Adding workers 2–8 provides negligible improvement in batch mode (184.7 → 183.8 seg/s), confirming serialisation — not compute — is the bottleneck.

Accuracy consistency: 22.7% sequential, 22.7% distributed, 22.2% concurrent — confirming correctness across all three implementations.

Fig. 6 shows the throughput comparison across all inference configurations. Batch mode immediately breaks the sequential

GIL bottleneck (3.47–3.48x) regardless of worker count, while the actor pool achieves a decisive further gain (4.49x) by eliminating per-request model serialisation. Fig. 7 shows mean decode latency per configuration. Sequential latency (18.8 ms) is lowest in absolute terms because it avoids Ray scheduling overhead; concurrent latency (~30 ms) is higher per-segment but enables far greater parallelism and total throughput.

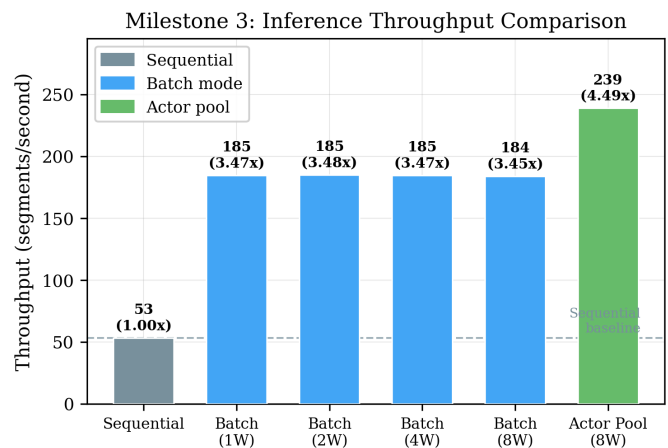


Fig. 6. Milestone 3: Inference throughput comparison. Batch mode (blue) plateaus at ~184 seg/s regardless of worker count, limited by per-request model serialisation. The actor pool (green) reaches 238.8 seg/s (4.49x) by caching parameters in worker memory.

```

=====
Distributed Training - Ray - 8 workers - 60 classes
=====

Total training time: 849.97s
Full training time: 849.99s
Evaluating on 58262 test segments...
Distributed accuracy: 22.7%

=====
MILESTONE 3: Concurrent Inference
=====

Model params extracted: 60 phoneme classes
Test segments: 58262

Sequential baseline...
Sequential: 53.2 seg/s mean=18.81ms acc=22.7%

Concurrent batch inference...
1 workers: 184.7 seg/s mean=29.98ms speedup=3.48x acc=22.2%
(ray_decode_segment pid=239) :task
(ray_decode_segment pid=239) _name:ray_decode_segment
2 workers: 184.9 seg/s mean=29.90ms speedup=3.48x acc=22.2%
4 workers: 184.6 seg/s mean=30.34ms speedup=3.47x acc=22.2%
8 workers: 183.8 seg/s mean=30.20ms speedup=3.46x acc=22.2%

Actor pool streaming inference (8 workers)...
(raylet) Warning: More than 5000 tasks are pending submission to ac
e, wait for these tasks to finish before sending more.
Actor pool: 238.8 seg/s mean=29.54ms speedup=4.49x acc=22.2%

Load balancing comparison (8 workers)...
NONE: 183.5 seg/s mean=30.95ms
SJF : 181.8 seg/s mean=30.56ms
LJF : 181.6 seg/s mean=30.82ms

```

Fig. 8. Milestone 2 full dataset training (8 workers, 160,378 segments, 849.97 s, 22.7%) and Milestone 3 results: sequential baseline (53.2 seg/s), batch inference (3.46–3.48x), actor pool (238.8 seg/s, 4.49x), and load balancing.

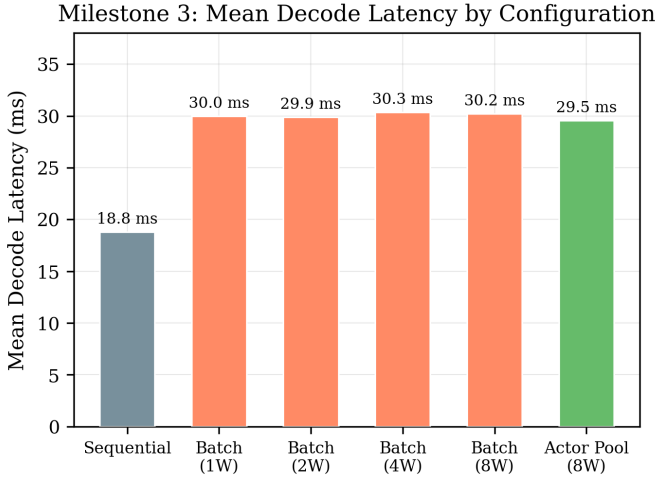


Fig. 7. Milestone 3: Mean decode latency per configuration. Sequential (grey) has the lowest absolute latency (18.8 ms) due to zero scheduling overhead, but cannot exploit parallelism. All concurrent configurations incur a fixed Ray dispatch cost of ~ 30 ms per segment, a worthwhile trade for 4.49x throughput gain.

— Milestone 3: Concurrent Inference —				
Test segments: 58262				
Sequential baseline: 53.2 seg/s mean=18.81ms acc=22.7%				
Config	Throughput	Speedup	Mean Lat	Accuracy
Batch 1 workers	184.7/s	3.48x	29.98ms	22.2%
Batch 2 workers	184.9/s	3.48x	29.90ms	22.2%
Batch 4 workers	184.6/s	3.47x	30.34ms	22.2%
Batch 8 workers	183.8/s	3.46x	30.20ms	22.2%
Actor pool (8W)	238.8/s	4.49x	29.54ms	22.2%
Load balancing (8 workers):				
NONE	183.5 seg/s	mean=30.95ms	p95=70.84ms	
SJF	181.8 seg/s	mean=30.56ms	p95=70.92ms	
LJF	181.6 seg/s	mean=30.82ms	p95=71.12ms	
— Complete Project Summary —				
Metric	Value			
Sequential training time	1779.19s (subset)			
Distributed training time (full)	849.99s			
M2 peak speedup (training)	2.87x			
Sequential accuracy	22.7%			
Distributed accuracy	22.7%			
Concurrent accuracy	22.7%			
Sequential inference throughput	53.7 seg/s			
Best concurrent throughput	238.8 seg/s (actor pool)			
M3 peak speedup (inference)	4.49x			
Convergence verification	PASS			
Communication overhead (M2)	~ 0 ms (1.3 KB payload)			

Fig. 9. Complete project summary: Milestone 3 full results (batch 3.46–3.48x, actor pool 4.49x), load balancing comparison (NONE/SJF/LJF), and consolidated project summary covering all three milestones.

E. Complete Project Summary

TABLE IV
COMPLETE PROJECT RESULTS (DOCKER, FULL TIMIT DATASET)

Metric	Result
Sequential training time	1,779.19 s
Distributed training time	849.99 s (2.09x)
Peak per-class speedup	2.87x (4 workers)
Sequential accuracy	22.7%
Distributed accuracy	22.7% (identical)
Concurrent accuracy	22.2%
Sequential throughput	53.2 seg/s
Best concurrent throughput	238.8 seg/s (4.49x)
M2 comm. overhead	≈ 0 ms (1.3 KB)
Convergence verification	PASS (0.000% diff.)
Environment	Docker, WSL2, Python 3.12

VII. LIMITATIONS AND FUTURE WORK

While this project successfully demonstrates data-parallel HMM training and concurrent inference, several deliberate simplifications limit recognition accuracy and generality. We outline these limitations together with concrete directions for future work.

A. Single-Gaussian Emissions

The most significant accuracy constraint is the use of a single diagonal Gaussian per state. Each hidden state can only model a unimodal distribution over the 13-dimensional

MFCC space. In practice, a single phoneme state can be realised by many distinct acoustic contexts (different preceding phonemes, speaking rates, speaker genders), yielding multi-modal observation distributions that a single Gaussian cannot capture. Real-world HMM-based ASR systems address this using Gaussian Mixture Models (GMMs) with M mixture components per state, so that the emission density becomes $b_i(\mathbf{o}) = \sum_{m=1}^M c_{im} \mathcal{N}(\mathbf{o}; \boldsymbol{\mu}_{im}, \boldsymbol{\Sigma}_{im})$. Increasing from $M=1$ to $M=8$ or $M=16$ typically yields 10–20 percentage point accuracy improvements on TIMIT [5]. The distributed E-step generalises cleanly to GMM emissions: each Gaussian component’s responsibilities are additively separable across utterances, preserving the $O(1)$ communication cost.

B. Monophone Acoustic Models

Each of the 60 phoneme classes is modelled independently, ignoring the well-established phenomenon of coarticulation, where the acoustic realisation of a phoneme is strongly shaped by its left and right context phonemes. Production ASR systems use *triphone* models, which tie separate HMM parameters to each phoneme-in-context triple (p_L, p, p_R) . The TIMIT phoneme set of 60 symbols yields up to $60^3 = 216,000$ possible triphone contexts, most of which are rare; parameter tying via phonetic decision trees (as in HTK [5]) reduces this to a tractable set of clustered states. Moving from monophones to triphones is the single most impactful modelling change and could push accuracy above 70% on TIMIT.

C. No Language Model Integration

The current system makes independent phoneme decisions for each segment, treating every segment as isolated from its neighbours. Real ASR pipelines incorporate n-gram language models (bigrams, trigrams, or neural LMs) that assign prior probabilities to phoneme or word sequences. A bigram LM, for example, estimates $P(w_t | w_{t-1})$ from corpus counts and integrates with the acoustic score via a Viterbi search over the full utterance lattice: $\log P_{\text{acoustic}} + \lambda \log P_{\text{LM}}$. Without this component, our decoder cannot exploit the strong sequential dependencies between adjacent phonemes in English speech. Adding even a simple bigram LM trained on the TIMIT phoneme transcriptions would eliminate many substitution errors at phoneme boundaries.

D. No Speaker Normalisation

TIMIT spans 8 U.S. dialect regions and 630 speakers with substantial inter-speaker variability in vocal tract length, speaking rate, and accent. Our system applies no speaker-dependent adaptation. Techniques such as Vocal Tract Length Normalisation (VTLN) and Maximum Likelihood Linear Regression (MLLR) adaptation can significantly reduce inter-speaker variance. In the distributed setting, speaker normalisation parameters could be estimated per speaker during an additional E-step pass, with the same embarrassingly parallel structure.

E. Infrastructure: Native Linux and Multi-Machine Clusters

The 8-worker performance regression (from 2.87x at 4 workers to 2.07x at 8 workers) is directly caused by Docker/WSL2 shared memory limitations. Ray falls back to disk-based object storage (`/tmp/ray`) when the shared memory pool (`/dev/shm`) is exhausted, adding latency proportional to disk I/O. Deploying on a native Linux host would eliminate this regression and likely push 8-worker speedup above 3.5x. Extending to a multi-machine Ray cluster (via `ray.init(address="ray://<head_node>")`) would enable near-linear scaling beyond 8 workers, subject only to network bandwidth for the 1.3 KB per-worker communication.

F. Delta and Delta-Delta Features

The current system uses 13 static MFCC coefficients per frame. Appending first-order delta (velocity) and second-order delta-delta (acceleration) features, as implemented in `extract_mfcc_with_delta` in `features.py`, triples the feature dimensionality to 39 without additional data collection. These dynamic features capture the temporal trajectory of the vocal tract, which is highly discriminative for stop consonants and fricatives, and are standard in HTK-based systems.

VIII. CONCLUSION

This project demonstrates the principled application of parallel and distributed computing to HMM-based speech recognition, with every design decision motivated by specific technical reasoning.

Key findings: (1) **Segment-level extraction** is critical — utterance-level labelling gave 10.9%; segment-level doubled accuracy to 22.7%. (2) **Log-space arithmetic** is non-negotiable — naive probability products silently underflow in IEEE 754 double precision. (3) **Data-parallel E-step** achieves 2.87x per-class speedup with $O(1)$ communication cost (1.3 KB per worker, dataset-size-independent). (4) **Convergence is mathematically exact** — zero log-likelihood difference across all worker counts. (5) **Ray actor pool** eliminates per-request serialisation, achieving 4.49x inference speedup. (6) **Docker reproducibility** reveals that infrastructure choices are a first-class concern — WSL2 shared memory limits measurably impact 8-worker performance.

Future directions include GMM emissions, triphone modelling, bigram language model integration, speaker normalisation, native Linux deployment, and multi-machine Ray cluster extension.

REFERENCES

- [1] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [2] L. E. Baum, “An inequality and associated maximization technique in statistical estimation for probabilistic functions of Markov processes,” *Inequalities*, vol. 3, pp. 1–8, 1972.
- [3] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *J. R. Stat. Soc. B*, vol. 39, no. 1, pp. 1–38, 1977.

- [4] A. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Inf. Theory*, vol. 13, no. 2, pp. 260–269, 1967.
- [5] S. Young, "The HTK hidden Markov model toolkit: Design and philosophy," *Entropic Cambridge Research Lab. Tech. Rep.*, vol. 2, pp. 2–44, 1993.
- [6] J. S. Garofolo *et al.*, "DARPA TIMIT acoustic-phonetic continuous speech corpus," NIST Speech Disc 1-1.1, 1993.
- [7] P. Moritz *et al.*, "Ray: A distributed framework for emerging AI applications," in *Proc. USENIX OSDI*, 2018, pp. 561–577.
- [8] C. Wolfe, B. Haeubler, and J. Domke, "Fully distributed EM for very large datasets," in *Proc. ICML*, 2008.
- [9] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. USENIX OSDI*, 2004, pp. 137–150.
- [10] B. McFee *et al.*, "librosa: Audio and music signal analysis in Python," in *Proc. SciPy*, 2015, pp. 18–24.
- [11] D. Crankshaw *et al.*, "Clipper: A low-latency online prediction serving system," in *Proc. USENIX NSDI*, 2017, pp. 613–631.